VULNERABILITY ASSESSMENT AND PENETRATION TESTING REPPORT

Target Application: OWASP Juice Shop

(Local Host)

Prepared By: Shashank

Internship Program: Future Interns – Cyber Security

Internship 2026

Testing Type: Black Box Testing

Tools used:

- OWASP ZAP (Passive Scanning & Vulnerability Detection)
- Browser Developer Tools (Network & Header Inspection)

Date: 26 February 2026

Introduction

Purpose & Target Application

This report presents the findings of a Vulnerability Assessment and Penetration Testing (VAPT) conducted on OWASP Juice Shop, running in a controlled local environment.

Objective of Testing

The objective of this assessment was to **identify security vulnerabilities, evaluate associated risks, and provide remediation recommendation** to improve the security posture of the application

Context & Environment

Testing was performed as part of the **Future interns cyber-Security Internship task-1.** All testing activities were conducted safely in a local environment; no live production systems were targeted.

During testing three vulnerabilities were identified: **SQL injection, CORS misconfiguration, and Open Redirected.** Details analysis and recommendations are provided in the following sections

Scope & Methodology

Scope

The scope of this assessment was limited to the **OWASP Juice Shop application installed locally.** Only the web application running in the controlled environment was tested. External system, live production servers, and third-party integration were not included in this assessment

Methodology

The testing methodology followed a **Black box Testing approach,** where the application was assessed without prior Knowledge of its internal code. The following steps were performed:

1. **Passive vulnerability Scanning using OWASP ZAP** to detect common vulnerabilities.

2. **Manual testing using Browser Developer Tools(F12),** inspecting network requests and application headers.

3. **Verification of identified issues** to confirm vulnerabilities and assess potential impact

This methodology ensures that all key application vulnerabilities were identified safely and efficiently within the local environment.

Vulnerabilities

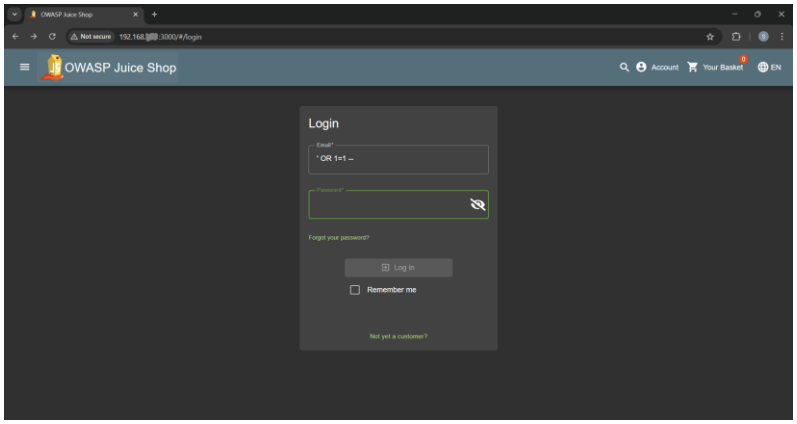
SQL Injection – Login Page

Description:

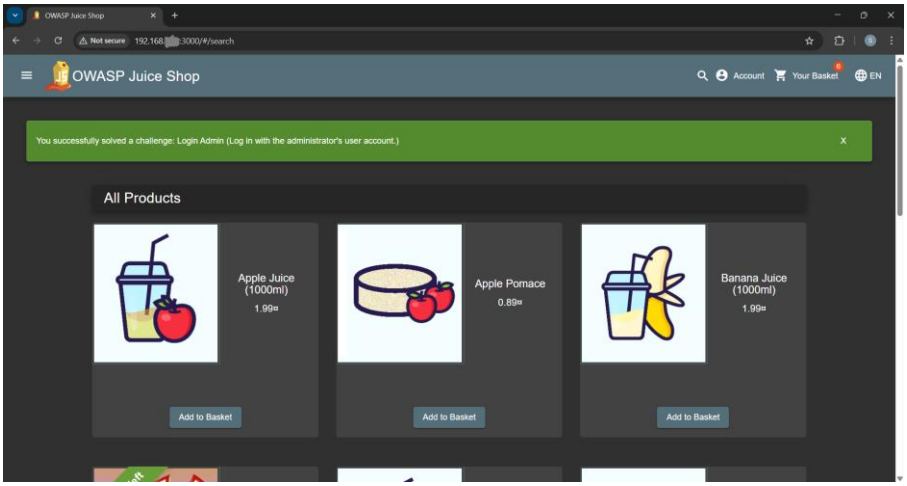
SQL Injection was found on the login page in the email input field. Improper input validation allows an attacker to inject SQL queries, which may expose sensitive data from the database.

Risk Level: High

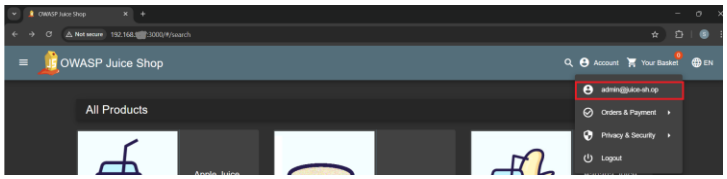
Proof of concept (PoC):



"Injected ' OR 1=1 -- in the email field to test SQL injection."



Login done successfully to admin account.



"Bypassed login validation and accessed the admin panel."

Remediation:

To eliminate SQL Injection, implement parameterized queries or prepared statement in the login form. Additionally, validate and sanitize all user inputs to prevent unauthorized database access.

Open Redirect – Checkout Page (Other Payment → Spreadshirt (US))

Description:

While testing the checkout page, the "other payment" option was selected. When choosing **Spreadshirt US,** the application accepts a redirect URL parameter without proper validation.

This allows an attacker to redirect users to any external website, which can be used for phishing or malicious purpose.

This vulnerability was detected in the **ZAP traffic scanner,** flagged as **High alert**

Risk Level: High

Proof of Concept (PoC):

"Checkout page showing the 'other payment' option with Spreadshirt US selected, where the redirect parameter is accepted"

Off-site Redirect

URL: http://192.168.[redacted]3000/redirect?to=http://shop.spreadshirt.com/juiceshop

Risk: High

Confidence: Medium

Parameter: to

Attack:

Evidence:

CWE ID: 601

WASC ID: 38

Source: Passive (10028 - Off-site Redirect)

"ZAP scanner flagged a high alert for open Redirect when the Spreadshirt US redirect was triggered, confirming the vulnerability."

Remediation:

To eliminate this open Redirect vulnerability, validate all redirect URLs in the checkout workflow to allow only trusted internal application URLs. Reject or sanitize any external or untrusted URLs. Implement strict server-side validation of redirect parameters to ensure users cannot be redirected to malicious websites, protecting both user security and application reputation.

CORS Misconfiguration – Initial Page Request

Description

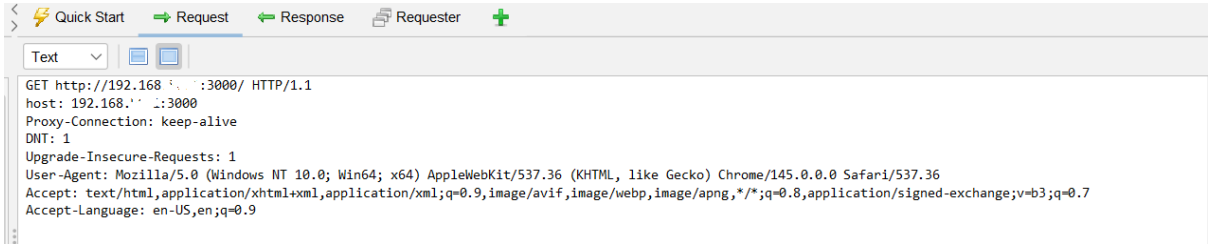
While observing the HTTP traffic generated by OWASP ZAP during the initial request to the target application, it was discovered that the application's response header allow overly permissive **cross-origin resource sharing.**

The ZAP scanner flagged this as a potential vulnerability, indicating that certain headers (e.g., Access-Control-Allow-Origin) are misconfigured, permitting requests from any origin.

This type of misconfiguration can allow a malicious website to make unauthorized request to the application on behalf of a victim user, leading to sensitive data disclosure or unintended actions.

Risk level: Medium

Proof of concept (PoC):



"Initial request sent to the Juice Shop application, captured in ZAP traffic, showing headers sent by the browser."

"Response headers captured in ZAP, showing `Access-Control-Allow-Origin: *`, indicating a **CORS** misconfiguration vulnerability."

Remediation:

Restrict the `Access-Control-Allow-Origin` header to only trusted domains instead of using *.

Validate the `Origin` header server-side and allow requests only from authorized front-end domains

Ensure related headers like

Access-Control-Allow-Methods and Access-Control-Allow-Credentials are properly configured.

Re-test to confirm the misconfigured is resolved.

Summary of Findings

vulnerability	Risk Level	Remediation
SQL Injection	High	Input validation/ Parameterized queries
Open Redirect	High	Restrict redirect URLs
CORS Misconfiguration	Medium	Restrict allowed origins / validate origin

Conclusion:

"During the security assessment of the Juice Shop application, several vulnerabilities were identified, including **SQL Injection, Open Redirect, CORS misconfiguration.** The application demonstrates a moderate security risk and should implement the recommended remediations to improve its security posture."

- Regularly update and patch the application
- Conduct periodic security testing
- Implement secure coding practices
- Restrict access to sensitive endpoints.